

ESP32-CAM Monitoring System

Overview

A low-power, automated video monitoring system built using an ESP32-CAM. The system captures video at regular intervals, stores it locally as a failsafe, and syncs to a centralized Django server for user access via a secure web dashboard.

Phase 1: Planning and Architecture

The foundational architecture defines the hardware behavior, data flow, and user experience.

Hardware Specifications

- **Capture Interval:** Wakes up every 15 minutes to record a 10-second video at CIF resolution and 30 frames per second.
- **Local Storage (The Backpack):** Saves files to a local SD card configured as a 3-day rolling buffer, automatically deleting files older than 72 hours to prevent overflow.
- **Network Delivery (The Mail Truck):** Utilizes standard HTTP POST requests to deliver every recorded video to the central server immediately after capture.
- **Device Identification:** Each ESP32 is assigned a unique Device ID hardcoded into its C++ firmware to link its uploads to a specific user account.

Server & Database Architecture

- **Central Hub:** A Linux server hosting the Django backend, the frontend website, and the database (built and managed natively using Python via Django's ORM).
- **Format Conversion:** The server receives raw .avi files from the ESP32 and processes them into .mp4 formats using FFmpeg for native web browser compatibility.
- **Server Storage Policy:** Maintains a synchronized 3-day rolling history of all user videos in the primary directory.
- **Cloud Protection:** A secondary, protected directory holds permanently saved videos. These videos bypass the 3-day deletion rule and are tied directly to the

user's profile.

Web Dashboard (Frontend UI)

- **Authentication:** Secure landing page for account creation and login (username and password only).
- **User Management:** Capabilities to edit credentials, delete accounts, and register new ESP32 cameras using unique Device IDs.
- **Profile Management:** Protected cloud videos appear under the user profile. Users can delete these videos at any time, which triggers a strict confirmation prompt ("Are you sure you want to delete [video name]?"). Confirming results in instant deletion.
- **Video Feed:** A 24-hour timeline of .mp4 videos (named username-date-time.mp4), resetting daily at 12:00 AM.
- **Video Interaction:** Clicking a video provides options to play the file directly on the website, download it to the local device, or allocate it to one of 10 available permanent "Cloud Save" slots.
- **Support:** A Help/FAQ page containing developer email contacts.

Phase 2: Component Testing

This phase validated the individual capabilities of the ESP32 hardware and the network infrastructure. A secure remote connection script was established to manage the Linux server using the command `ssh general182@100.98.214.116`.

The hardware testing phase consisted of seven isolated proof-of-concept test cases:

1. **Serial Heartbeat Test:** Opened a 115200 baud serial connection to verify the microcontroller was awake and communicating.
2. **Camera Initialization Test:** Assigned the AI Thinker GPIO pins and configured the camera for VGA resolution at 20MHz.
3. **SD Card Initialization Test:** Verified the SD_MMC library could successfully mount the physical storage card.
4. **Basic Photo Capture Test:** Captured a single JPEG frame buffer and saved it as `test_photo.jpg` to the SD card.

5. **Dual-Core Video Recording Test:** Implemented FreeRTOS tasks to capture a 10-second CIF 30 FPS video, utilizing `avi_stapler.h` to successfully generate the AVI file headers and data chunks.
6. **Wi-Fi Server Download Test:** Connected the ESP32 to a local Wi-Fi hotspot and hosted a temporary web server to allow direct video downloads from the SD card to a mobile browser.
7. **Webhook Upload Test:** Successfully pushed a recorded .avi file from the SD card to a remote URL using an HTTPClient POST request.

Phase 3: Core Development & Integration (In Progress)

This active phase focuses on merging the tested components into the final production system.

- **Firmware Integration:** Combining the deep sleep interval timer, the 3-day SD card rolling buffer logic, and the final HTTP POST delivery into a single ESP32 C++ script.
- **Server Backend:** Building the Python database models to handle user authentication, device routing, .mp4 conversion, and the 10-slot cloud storage allocation.
- **Frontend Development:** Constructing the HTML/CSS user interface, the 24-hour video dashboard, and account management views.
- **System Testing:** End-to-end verification ensuring a frame captured by the physical camera successfully traverses the network, converts formats, and displays on the user's secure web timeline.

Phase 4: Hardware Enclosure

The final deployment phase focuses on physical protection and aesthetics.

- **Chassis Design:** Sourcing or designing a custom 3D-printable housing for the ESP32-CAM and its power supply.
- **Assembly & Deployment:** Printing the chassis, assembling the hardware, and conducting real-world environmental field tests.